Comparative Study of Reduced Models for a Parametric Navier–Stokes Equation

Mesi Erina - 348365 Murariu Sara Magdalena - 349370

Abstract

This work presents a comprehensive numerical and comparative evaluation of three reduced-order modeling (ROM) methodologies applied to the parametric steady Navier–Stokes equations: projection-based POD-Galerkin, data-driven POD-NN, and mesh-free Physics-Informed Neural Networks (PINNs), using a high-fidelity Finite Element Full Order Model (FOM) as the numerical reference baseline. The numerical results reveal distinct engineering trade-offs across the paradigms.

The POD-Galerkin framework demonstrates the highest fidelity, achieving a mean relative full-state error under 1% (0.675%) alongside a substantial 427× speedup over the FOM baseline. Conversely, the POD-NN architecture completely bypasses the iterative Newton solver, delivering an instant forward-pass prediction with a 12.93% mean state error. Finally, the PINN approach highlights a radically alternative, mesh-free paradigm that embeds governing physical laws directly into the neural loss function. However, the baseline PINN achieved a poor mean relative error (108.68%) and was about ≈ 58 times slower than the FOM reference baseline.

Ultimately, this study confirms that while purely data-driven shortcuts provide exceptional speedups, projection-based frameworks remain the most stable and reliable methods for structured parametric fluid problems, whereas competitive physics-informed models demand highly specialized loss-balancing and training workflows.The complete code repository and project implementation can be found on GitHub at: https://github.com/SaraMurariu/Comparative-Study-of-Models-for-Parametric-Navier-Stokes-Equations.git.

1 Problem Formulation

The problem considered in this work is the stationary incompressible Navier–Stokes system on the two-dimensional unit square domain $\Omega = (0,1)^2$. Given a multi-query parameter vector $\mu = (\mu_0, \mu_1) \in \mathcal{P} = [0.1, 10.0] \times [1.0, 3.0]$, the objective is to find the velocity vector field $u = (u_x, u_y) : \Omega \to \mathbb{R}^2$ and the scalar pressure field $p : \Omega \to \mathbb{R}$ such that:

$$\begin{cases} -\mu_0 \nabla \cdot (\nabla u) + (\nabla u)u + \nabla p = f(x;\mu_1) & \text{in } \Omega, \\ (\nabla \cdot u) = 0 & \text{in } \Omega, \\ u = 0 & \text{on } \partial\Omega, \\ p(0,0) = 0 & \text{pressure normalization condition} \end{cases} \tag{1}$$

Here, μ_0 explicitly scales the viscous diffusion operator, while the parameter μ_1 governs the non-affine source term $f(x;\mu_1) = (f_1, f_2)$, which is defined component-wise as:

$$\begin{cases} f_1(x,y;\mu_1) = -(\mu_1^3\pi^2 \cos(\mu_1^2\pi x) - \mu_1^2\pi^2) \sin(\mu_1\pi y) \cos(\mu_1\pi y) + (\mu_1\pi \cos(\mu_1\pi x) \cos(\mu_1\pi y)), \\ f_2(x,y;\mu_1) = -(-\mu_1^3\pi^2 \cos(\mu_1^2\pi y) + \mu_1^2\pi^2) \sin(\mu_1\pi x) \cos(\mu_1\pi x) - (\mu_1\pi \sin(\mu_1\pi x) \sin(\mu_1\pi y)) \end{cases} \tag{2}$$

The first relation describes the balance of momentum, where the term $(\nabla u)u$ introduces a quadratic transport nonlinearity. The second relation enforces mass conservation via the incompressibility constraint. To eliminate the algebraic singularity stemming from the fact that pressure is uniquely defined only up to an additive constant, a single-point condition $p(0,0) = 0$ is explicitly imposed at the origin.

The continuous weak formulation is obtained by multiplying the strong equations by suitable test functions and integrating over the domain, shifting the problem from a differential to an integral setting. The natural functional spaces for the continuous problem are $V = [H_0^1(\Omega)]^2$ for velocity and $Q = L^2(\Omega)$ for pressure. The weak integrated problem reads:

find $(u, p) \in V \times Q$ such that:

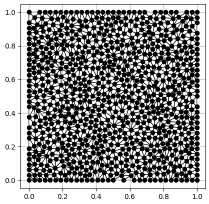
$$\begin{cases} a_\mu(u,v) + c(u,u,v) + b(v,p) = F_\mu(v) \quad \forall v \in V, \\ b(u,q) = 0 \quad \forall q \in Q, \end{cases} \tag{3}$$

where the bilinear, trilinear, and forcing forms are defined as:

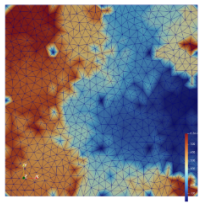
$$a_\mu(u,v) = \mu_0 \int_\Omega \nabla u : \nabla v \, dx, \quad c(u,w,v) = \int_\Omega (u \cdot \nabla) w \cdot v \, dx, \quad b(v,p) = -\int_\Omega p \nabla \cdot v \, dx$$

$$F_\mu(v) = \int_\Omega f(x;\mu_1) \cdot v \, dx$$

The goal is to solve the system for different parameter values and then compare a high-fidelity finite element solution (FEM) with reduced-order and neural approximations.



(a) Generated mesh



(b) Mesh visualized in Paraview

2 Finite Element Model (FOM)

2.1 Discretization

The continuous spatial domain is discretized via a finite element mesh (Figure 1a) with a characteristic mesh size parameter $h = 0.001$ and a total domain area of $|\Omega| = 1$. This decomposition is the central idea of FEM: instead of looking for the exact solution in the whole continuous domain, the method builds a finite-dimensional approximation by combining many simple local functions.

The infinite-dimensional spaces V and Q are replaced with stable, inf-sup conforming, finite-dimensional subspaces $V_h \subset V$ and $Q_h \subset Q$. Velocity and pressure are locally approximated on each individual triangular element by linear combinations of simple polynomial basis functions (φ_i, ψ_j) scaled by the coefficient vectors (U_i, P_j):

$$u_h(x;\mu) = \sum_{i=1}^{N_u} U_i(\mu)\varphi_i(x), \quad p_h(x;\mu) = \sum_{j=1}^{N_p} P_j(\mu)\psi_j(x)$$

This yields a global discrete state vector $U_h(\mu) = [U(\mu), P(\mu)]^T \in \mathbb{R}^{N_h}$ with a total dimension of $N_h = 6761$ degrees of freedom.

2.2 Validation

Due to the quadratic nonlinearity of the convective term $(u \cdot \nabla)u$, the discrete algebraic system cannot be solved via direct matrix inversion. Instead, we formulate the system in its monolithic residual form, $R_h(U_h;\mu) = 0$, and resolve it iteratively using the Newton–Raphson method. At each iteration k, the solver linearizes the curved physics surface by computing the system Jacobian matrix $J_h(U_h^{(k)};\mu)$, which acts as a local multi-dimensional tangent hyperplane touching the residual function at the current state guess $U_h^{(k)}$. The localized linear correction step, $\delta U_h^{(k)} = U_h^{(k+1)} - U_h^{(k)}$, is determined by finding where this tangent hyperplane crosses zero, matching the classical linear system:

$$J_h(U_h^{(k)};\mu)\delta U_h^{(k)} = -R_h(U_h^{(k)};\mu) \tag{4}$$

Starting from an initial linear Stokes solution guess $U_h^{(0)}$, this system is solved sequentially until the norm of the residual vector drops below a strictly defined tolerance threshold.

Before utilizing the Full Order Model (FOM) as the ground-truth baseline, the numerical framework is rigorously verified using the Method of Manufactured Solutions (MMS). Predefined, smooth analytical sine and cosine fields (5) that natively satisfy the homogeneous Dirichlet boundary conditions were passed into the system. By substituting these fields into the Navier–Stokes equations, the corresponding forcing term was analytically derived.

$$\begin{cases} u_x(x, y; \mu_1) = \dfrac{1}{2}\mu_1 \sin^2(\pi x) \sin(\pi y) \cos(\pi y), \\ u_y(x, y; \mu_1) = -\dfrac{1}{2}\mu_1 \sin^2(\pi y) \sin(\pi x) \cos(\pi x), \\ p(x, y; \mu_1) = \mu_1 \sin(\pi x) \sin(\pi y). \end{cases} \tag{5}$$

The finite element solver was executed across five test parameter pairs spanning the spatial and parametric domain boundaries, yielding the exact diagnostic tracking summarized in Table 1

Table 1: Numerical results for different parameter values.

μ_0	μ_1	err_velocity	err_u_x	err_u_y	err_p	Newton it.	Rel. increment	Converged
0.5	1.2	0.015584	0.010756	0.011277	0.062143	3	6.98×10^{-8}	True
1.0	2.0	0.015137	0.010715	0.010693	0.083573	3	7.82×10^{-8}	True
3.0	2.5	0.015023	0.010829	0.010413	0.162693	3	1.57×10^{-8}	True
7.0	1.5	0.015018	0.010883	0.010350	0.290898	3	5.44×10^{-10}	True
10.0	3.0	0.015019	0.010896	0.010338	0.366675	3	1.94×10^{-9}	True

The global error columns (`err_velocity`, `err_u_x`, `err_u_y`, `err_p`) represent the relative L^2 errors. Using a Newton tolerance of 10^{-6} on the relative increment ratio, the stable convergence within exactly 3 iterations successfully validates our FOM model.

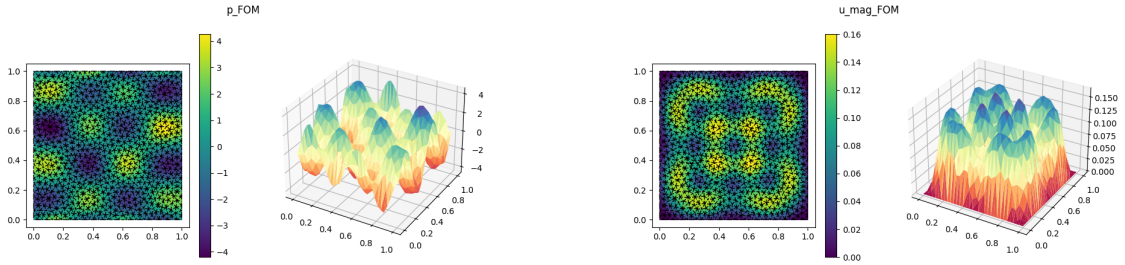
2.3 High fidelity reference solution

With the core numerical machinery verified, the framework switches to evaluating the true physical system in the discretized, full-order domain. As previously during the validation, the Newton method is applied to solve the non-linear system.

The iterations are initialized from a uniform zero condition field ($U_h^{(0)} = \mathbf{0}$).

The solver executes this high-fidelity baseline across the physical parameter space defined by the continuous boundaries $\mathcal{P} = [0.1, 10.0] \times [1.0, 3.0]$, sampled using a uniform distribution generating 100 distinct parameter pairs.

Using a Newton convergence tolerance of 10^{-6} on the relative increment ratio and and maximum number of iterations equal to 10, the framework consistently achieves rapid convergence within 3 iterations.



(a) Pressure field

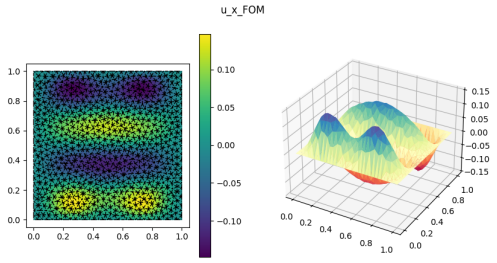
(b) Velocity magnitude.

Figure 2

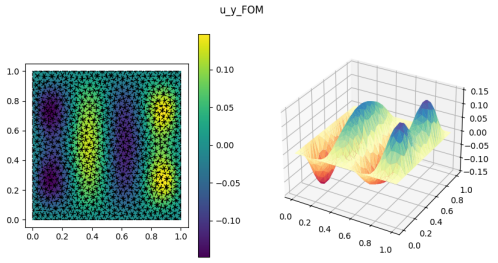
2.4 Results

The pressure field in Figure 2 shows an oscillatory pattern with positive and negative regions distributed over the domain. The velocity magnitude is zero on the boundary, consistently with the Dirichlet condition, and reaches its largest values inside the domain.

The horizontal and vertical velocity components Figure 3, u_x and u_y, can be either positive or negative. Their sign shows the direction of the flow along each axis. When combined, they give the velocity magnitude, which is always non-negative and shows where the flow is faster, regardless of its direction.



(a) Horizontal velocity



(b) Vertical velocity

Figure 3

3 POD-GALERKIN

The Proper Orthogonal Decomposition (POD) combined with a Galerkin projection represents a model order reduction strategy. Its main objective is to reduce the computational complexity of the Full Order Model (FOM) by compressing the massive $N_h = 6761$ dimensional system down to a low-dimensional reduced-order model.

3.1 Proper Orthogonal Decomposition (POD)

The process begins offline by gathering data across the continuous parameter space $\mathcal{P} = [0.1, 10] \times [1, 3]$. The parameter space is uniformly sampled to construct an ensemble of $M = 100$ parameter configurations (same as previous FOM computations). The full high-fidelity Newton solver is executed for each parameter pair in this ensemble, and the resulting discrete solution vectors are stacked side-by-side to construct the Snapshot Matrix $S \in \mathbb{R}^{N_h \times M}$:

$$S = \begin{bmatrix} | & | & & | \\ U_h(\mu^{(1)}) & U_h(\mu^{(2)}) & \cdots & U_h(\mu^{(100)}) \\ | & | & & | \end{bmatrix}$$

To extract the most energetic spatial features from this dataset, a Singular Value Decomposition (SVD) is performed on the snapshot matrix, yielding $S = U\Sigma V^T$. The columns of the orthogonal matrix $U \in \mathbb{R}^{N_h \times N_h}$ represent the left singular vectors (POD modes), while the diagonal matrix Σ contains the singular values ordered by decreasing magnitude ($\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_M$).

By applying a relative energy tolerance ($tol = 10^{-6}$) criterion, the discrete space is truncated, retaining only the first $N \ll N_h$ columns of U:

$$\Phi_N = \begin{bmatrix} | & | & & | \\ \phi_1 & \phi_2 & \cdots & \phi_N \\ | & | & & | \end{bmatrix}$$

whose span is used to form the Reduced Basis Matrix V_N. In this framework the effective reduced dimension is $N = 49$ which retain more than 99.99% of the snapshot energy according to the selected tolerance.

3.2 Galerkin Projection

With the reduced basis matrix Φ_N built, every high-dimensional solution can be approximated as a simple linear combination of these $N = 49$ spatial modes. Instead of solving for the original finite element degrees of freedom, the unknowns become the reduced coefficients $\mathbf{a}(\mu) \in \mathbb{R}^N$

$$U_h(\mu) \approx V_N \mathbf{a}(\mu) = \sum_{i=1}^{N} a_i(\mu)\phi_i$$

where ϕ_i are the POD basis functions. To find the coefficients $\mathbf{a}(\mu)$, we perform a Galerkin Projection, which forces the new system to be orthogonal to our generated reduced space by multiplying by Φ_N^T:

$$R_N(a(\mu); \mu) = \Phi_N^T R_h(\Phi_N a(\mu); \mu) = 0$$

This system remains non-linear because of the convective term. Consequently, Newton's method is applied again but to a system of only 49 unknowns instead of 6761. An important computational issue is the convective term.

Since the convection is quadratic in the velocity, it can be precomputed in reduced coordinates. If the reduced velocity is written as the POD expansion formula ($u_N = \sum a_i \phi_i$), then the reduced convective factor is of the form

$$[c_N(a)]_i = c\left(\sum_{j=1}^{N} a_j \phi_j,\ \sum_{k=1}^{N} a_k \phi_k,\ \phi_i\right) \quad \rightarrow [c_N(a)]_i = \sum_{j=1}^{N}\sum_{k=1}^{N} a_j a_k\ \mathbf{c}(\phi_\mathbf{j}, \phi_\mathbf{k}, \phi_\mathbf{i})$$

The third-order component $c(\phi_j, \phi_k, \phi_i)$ depends solely on the spatial POD modes and can be precomputed entirely offline; without this strategy, since velocity affects itself, we would need to recompute each time the same original, full-mesh system.

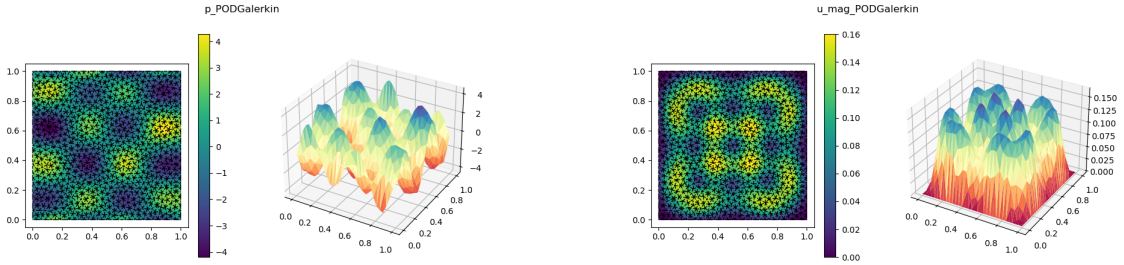
A further practical issue concerns the reduced forcing term. Because the forcing term is not easily available in affine separated form (i.e., it cannot be separated from a parameter, in our case μ_1) the online computation of $f_N(\mu_1)$ may still require operations at the full finite element dimension because the source term cannot be computed offline.

A classical strategy to recover an efficient approximation is the Empirical Interpolation Method (EIM), which constructs a separated approximation of the non-affine term by selecting interpolation basis functions and interpolation degrees of freedom. In this work, the reduced forcing vector is approximated by a small one-hidden-layer neural network:

$$\mu_1 \longrightarrow \boxed{\text{Neural Network}} \longrightarrow f_N(\mu_1)$$

For each training sampled value of μ_1, the full FEM right-hand side is assembled and projected onto the reduced basis: $f_N(\mu_1) = \Phi_N^T f_h(\mu_1)$; the model thus learns the relationship between a given parameter vector (parameter sampled from usual uniform distribution) and its corresponding reduced source term. The implemented model is a shallow neural network with a fixed random hidden layer, hyperbolic tangent activation function and trainable output weights fitted by ridge regression. This data driven reduction approach was preferred due to its semplicity.

The POD-Galerkin model was evaluated on 10 out-of-sample test parameter pairs (4).



(a) Pressure field

(b) Velocity magnitude.

Figure 4

3.3 Results

Qualitatively, the POD-Galerkin plots are almost indistinguishable from the high-fidelity FOM reference fields. The structured oscillatory pattern of the pressure field, the internal fluid structures, and the zero-velocity boundary conditions are perfectly preserved. This high accuracy is expected because the $N = 49$ global basis functions are built directly from high-fidelity physics snapshots.

The averaged performance metrics demonstrate sufficient accuracy and massive computational savings: "'latex

- **Mean Relative Error:** 6.75×10^{-3}, corresponding to a mean error of 0.675% over the full state, with a maximum error of 2.94%.
- **Average Online ROM Time:** 6.23×10^{-4} s, or 0.623 ms, demonstrating nearly instantaneous execution.
- **Computational Speedup:** Approximately 664× faster than the FOM on average.

4 POD-NN

The POD-NN (Proper Orthogonal Decomposition Neural Network) method is a data-driven reduced order model that combines the POD approaches with neural networks.

The main objective of the POD-NN is to completely substitute the iterative Newton solver. Instead of performing iterative Newton iterations to find the coefficients, we train a Multi-Layer Perceptron (MLP) to learn the mapping $\mu = (\mu_0, \mu_1) \rightarrow a(\mu)$.

The POD basis is still constructed exactly as in the POD-Galerkin approach. The neural network only replaces the online computation of the reduced coefficients.

4.1 Training

During the offline phase, we use the exact same 100 parameter pairs (training_set) as earlier to obtain the snapshots; POD is then applied to these snapshots by projecting the high-fidelity solutions onto the basis matrix ($\mathbf{a}^{(m)} = \Phi_N^T U_h^{(m)}$) to extract the true 49 coefficient targets. The neural network is then trained:

$$\mu = \begin{bmatrix} \mu_0 \\ \mu_1 \end{bmatrix} \longrightarrow \boxed{\text{Layer 1: 128}} \longrightarrow \boxed{\text{Layer 2: 128}} \longrightarrow \boxed{\text{Layer 3: 64}} \longrightarrow \mathbf{a}(\mu) = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_{49} \end{bmatrix}$$

The network minimizes the standard Mean Squared Error (MSE), and the hidden activation functions are, as for the POD-Galerkin, hyperbolic tangents. The network is trained with the Adam optimizer, using 5000 epochs, learning rate of 10^{-3} and weight decay of 10^{-8}.

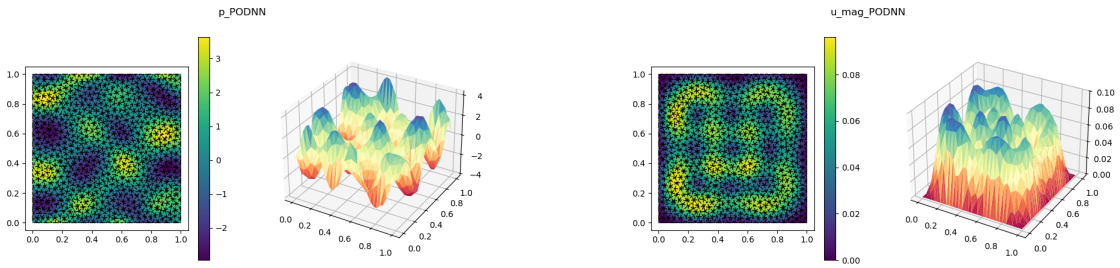
Once the network has been trained, generating a fluid simulation for a brand-new parameter condition is computationally inexpensive in this framework. Because the data was standardized before training (necessary step in any neural network training), when a new parameter μ is fed into the model, the network's raw output vector—denoted as $N_\theta(\mu)$—is also in this standardized scale, and must be converted to its original value:

$$a(\mu) = a_{\text{std}} \odot N_\theta(\mu) + a_{\text{mean}}$$

The final step is to project these numbers back onto the high-fidelity grid. The computer executes a single matrix-vector multiplication to combine the 49 predicted weights with the spatial basis functions (Φ_N):

$$U_{\text{PODNN}}(\mu) = \Phi_N a_{\text{PODNN}}(\mu)$$

The trained POD-NN model was evaluated on the same 10 out-of-sample test parameter configurations to directly compare its behavior against the full-order model and the POD-Galerkin framework (5)



(a) Pressure field

(b) Velocity magnitude.

Figure 5

4.2 Results

The POD-NN solution preserves the main qualitative structure of the FOM: the pressure still has alternating positive and negative regions, and the velocity magnitude remains zero on the boundary. However, the amplitude of the velocity magnitude is underestimated. The maximum value is close to 0.10, whereas the FOM and POD-Galerkin reach approximately 0.16. This means that the network captures the global spatial pattern but loses part of the intensity of the solution. For the performance metrics:

- **Mean Relative Error:** 0.1293, corresponding to a mean error of 12.93% over the full state, with a maximum error of 42.48%.

- **Average Evaluation Time:** 4.57×10^{-4} s, or 0.457 ms.
- **Computational Speedup:** Approximately 902× faster than the FOM on average, with speedups ranging from 90× to 836×.

Although POD-NN achieves an online computational cost comparable to POD-Galerkin, its prediction error increases from 0.675% to 12.93% . This drop in accuracy occurs because the POD-NN is completely blind to the underlying physics equations and only learns statistical patterns. However, it is also exceptionally fast: running a new parameter requires only a single, millisecond-level forward pass through the three-layer network, followed by a quick spatial matrix expansion. The trade-off between accuracy and computational effectiveness is clear in this example.

5 Physics-Informed Neural Network (PINN)

The Physics-Informed Neural Network (PINN) approach represents a paradigm shift from projection-based methods like POD-Galerkin and POD-NN. It is completely mesh-free and does not require an offline snapshot matrix or a reduced basis. Instead, a deep neural network is trained to approximate the continuous fluid fields directly as a function of space and parameters:

$$N_\theta(x, y, \mu) = \left(u_x^\theta(x, y, \mu), u_y^\theta(x, y, \mu), p^\theta(x, y, \mu)\right)$$

The input is four-dimensional (spatial coordinates x, y and physical parameters μ_0, μ_1), while the output is three-dimensional (velocity components u_x, u_y and pressure p). To ensure well-conditioned optimization across variables with highly differing numerical scales, all inputs are normalized to a uniform range of $[-1, 1]$ before entering the network.

5.1 Architecture

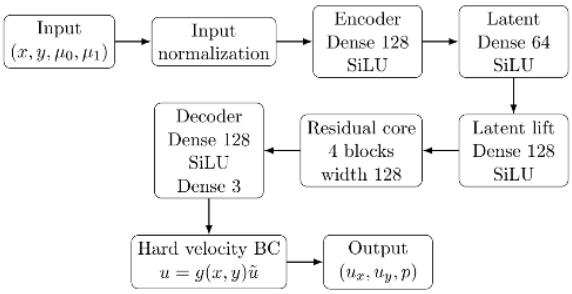


Figure 6: PINN architecture.

The PINN features an advanced encoder-residual–decoder fully connected topology:

- **Encoder** $(4 \to 128 \to 64)$**:** Maps the normalized inputs to a compact 64-dimensional latent representation using SiLU activation functions.
- **Residual Core:** Expands the latent vector back to 128 features and processes it through four residual blocks. The residual connections allow the network to learn incremental physical corrections and significantly improve training stability.
- **Decoder** $(128 \to 128 \to 3)$**:** Maps the processed features back to the physical variables (u_x, u_y, p).

To guarantee physical compliance at the boundaries, a hard boundary enforcement mechanism is embedded directly into the network's architecture for the velocity field. The raw velocity outputs of the model are multiplied by a distance-to-wall scaling factor:

$$g(x, y) = x(1 - x)y(1 - y)$$

Because $g(x, y) = 0$ everywhere on the boundary of the unit square $\partial\Omega$, the zero-velocity no-slip condition ($u^\theta = 0$) is satisfied by construction, removing the burden of learning boundary constraints from the optimizer. The raw pressure output is used directly ($p^\theta = p$).

5.2 Loss formulation

The spatial derivatives are computed directly from the network's computational graph to evaluate the strong form of the Navier-Stokes residuals: the momentum residuals (r_x^θ, r_y^θ) and the incompressibility/divergence residual (r_{div}^θ).

The network is optimized over 10,000 epochs using the same learning rate and weight decay, implementing Adam optimizer by minimizing a composite loss function evaluate across $N_{\text{interior}} = 4096$ and $N_{\text{boundary}} = 1024$ randomly sampled collocation points:

$$\mathcal{L}(\theta) = \lambda_{\text{PDE}}\mathcal{L}_{\text{PDE}} + \lambda_{\text{div}}\mathcal{L}_{\text{div}} + \lambda_{\text{b}}\mathcal{L}_{\text{boundary}} + \lambda_{\text{p}}\mathcal{L}_{\text{pressure\_anchor}}$$

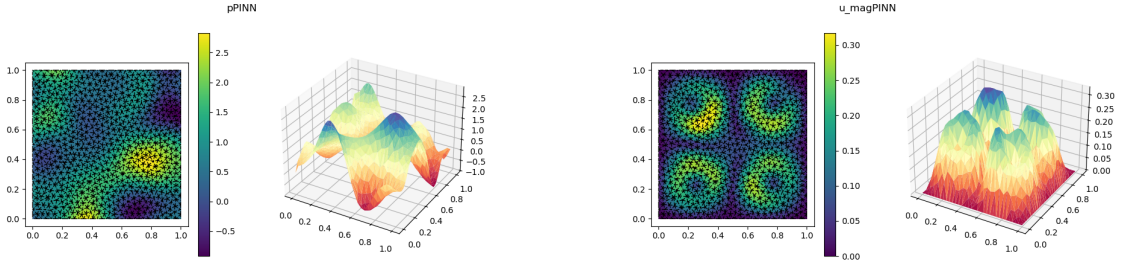
Where:

- **PDE Loss ($\mathcal{L}_{\text{PDE}}$):** Penalizes violations of momentum conservation within the interior of the domain.
- **Divergence Loss ($\mathcal{L}_{\text{div}}$):** Penalizes violations of the fluid incompressibility condition.
- **Boundary Loss ($\mathcal{L}_{\text{boundary}}$):** Enforces velocity consistency along the domain walls.
- **Pressure-Anchor Loss ($\mathcal{L}_{\text{pressure\_anchor}}$):** Penalizes the pressure value at the coordinate $(0, 0)$ to establish a reference pressure, since pressure is otherwise defined only up to an additive constant:

$$\mathcal{L}_{\text{pressure\_anchor}} = \frac{1}{N_p}\sum_{i=1}^{N_p} |p^\theta(0, 0, \mu_i)|^2$$

A grid search was performed over different combinations of loss weights. The selected configuration provided the lowest validation score of 13.137, balancing the momentum and incompressibility residuals while maintaining a stable optimization process. The optimal loss weights were after tuning were: $\lambda_{\text{PDE}} = 10$, $\lambda_{\text{div}} = 5$, $\lambda_{\text{b}} = 1$, and $\lambda_{\text{p}} = 1$.

The model was evaluated on the exact same 10 out-of-sample test parameters to ensure a perfectly fair benchmarking against the POD-Galerkin and POD-NN models (7).



(a) Pressure field

(b) Velocity magnitude.

Figure 7

5.3 Results

The PINN plots show that the model has learned some qualitative features of the solution, such as the vanishing velocity magnitude near the boundary and the presence of internal spatial structures. Nevertheless, the field amplitudes and spatial patterns differ significantly from the FOM. The pressure range is smaller and shifted with respect to the FOM, and the velocity magnitude reaches values around 0.30, which is almost twice the FOM scale. This indicates that the present PINN is not yet quantitatively reliable as a surrogate for the FOM.

Quantitatively, the final test highlight the limitations of the current PINN implementation:

- **Mean Relative Error:** 1.0868, with a maximum error of 1.3424 over the test parameter set.
- **Average Evaluation Time:** 24.34s.
- **Computational Speedup Factor:** 0.017× with respect to the Full Order Model (FOM), meaning that the current PINN implementation is approximately 59× slower than the FOM during inference.

From an engineering perspective, the PINN offers an attractive mesh-free framework, as it does not require finite element assembly, snapshot generation, reduced basis construction or Galerkin projection. Instead, the solution is approximated directly by a neural network through the governing equations. However, training a PINN is considerably more challenging than standard supervised learning. The optimization process must simultaneously satisfy multiple competing physical objectives, including momentum conservation, incompressibility, boundary conditions, and pressure normalization. Consequently, the choice of the loss weights λ has a significant impact on the quality of the final solution.

Despite the hyperparameter tuning, the current PINN implementation remains significantly less accurate than the projection-based reduced-order models. The mean relative error over the test set is 108.68%. Moreover, the average inference time is about 24.35 seconds, much slower than the FOM in this implementation. These results do not imply that PINNs are unsuitable in general; they show that the current training setup is not sufficient for this specific quantitative comparison

6 Comparison with FOM & conclusions

Table **??** summarizes the main quantitative results. All errors are relative errors with respect to the FOM solution. The FOM itself is not reported as an approximation method because it is the reference model.

Table 2: Comparison of the computational performance of the different methods.

Metric	**POD-Galerkin**	**POD-NN**	**PINN**
Mean $\boldsymbol{v}$ error (%)	0.1617	14.7659	72.79
Mean p error (%)	0.6754	12.9293	108.69
Mean FOM time (s)	1.0228	1.0163	0.42
Mean ROM time (s)	0.0029	0.0029	24.35
Mean speedup	426.93×	487.87×	0.017×

This work compared four modeling strategies for a parametric Navier–Stokes fluid problem:

- **Full Order Model (FOM):** Provides highly accurate and physically reliable results. However, its computational cost scales with the large 6,761-node mesh, making it unsuitable for real-time applications.
- **POD-Galerkin:** Offers the best overall balance between accuracy and computational speed. By projecting the governing equations onto a low-dimensional space of 49 modes and using a fast neural surrogate for the non-affine forcing term, it achieves errors 1% and an average speedup of approximately 427×. It is the preferred method when the original solver and governing equations are fully accessible.
- **POD-NN:** Provides a fast and non-intrusive black-box surrogate. It avoids iterative Newton loops by directly predicting the reduced coefficients from the input parameters. Although it is less accurate, with a state error of 12.93%, it is computationally efficient and easy to deploy when the original physics equations or solver cannot be modified.
- **PINN:** Uses a completely mesh-free approach and avoids snapshot data by including the strong form of the governing PDE directly in the loss function. However, the interaction between multiple physical loss terms makes the optimization process difficult. The baseline training was insufficient, resulting in a high state error of 108.68% and a large evaluation time.

Overall, POD-Galerkin stands out as the most reliable framework for this problem, POD-NN is the most efficient data-driven shortcut and PINNs remain a promising frontier that requires more advanced, adaptive training strategies.